

בלאק-ג'ק רב-משתתפים - תיק פרויקט

חלופת התמחות: הגנת סייבר ומערכות הפעלה

מסלול: הנדסת תוכנה 883589

1. מבוא

1.1 ייזום

1.1.1 תיאור ראשוני של המערכת

הפרויקט הוא משחק בלאק-ג'יק (21) רב-משתתפים על רשת בארכיטקטורת לקוח-שרת. מספר שחקנים מתחברים ממחשבים שונים לשרת מרכזי, ומשחקים יחד כנגד דילר ממוחשב.

מה שנבנה בסופו של דבר:

- שרת משחק - מקבל חיבורים, מנהל את חוקי המשחק, שומר פרופילי שחקנים ומחזיק הכול ב-SQLite על הדיסק.
- לקוח גרפי (Tkinter) - מסך התחברות/הרשמה, שולחן בלאק-ג'יק עם קלפים, כפתורי - שדה הימור, סרגל פרופיל וצ'אט בזמן אמת, Hit/Stand/Double.
- לוח בקרה לשרת (ServerApp) - חלון ניהול נפרד: רשימת שחקנים מחוברים, כפתור Kick לכל - שחקן, לוג צבעוני ושלב המשחק הנוכחי.
- סינון צ'אט עם (AI ChatModerator) - כל הודעת צ'אט עוברת דרך Gemini 2.5 Flash לפני - שידורה. אם המפתח לא מוגדר, המשחק עובד בלעדיו בלי בעיה.
- שכבת הצפנה - לחיצת יד RSA + Fernet: כל הודעה מוצפנת ומאומתת.
- שכבת הרשאות - סיסמאות מגובבות (PBKDF2), כל פעולה מחייבת שהשחקן מחובר ומאומת.
- למה בחרתי בבלאק-ג'יק: הוא פשוט מספיק בחוקיו כדי שהמיקוד יישאר בנושאי החלופה - תקשורת, ריבוי לקוחות, שמירת מצב והגנת סייבר - אבל מורכב מספיק כדי לדרוש מכונת מצבים, סנכרון בין שחקנים ולוגיקת קלפים אמיתית.

אתגרים שידעתי שיחכו לי:

- לתכנן פרוטוקול תקשורת שמתמודד עם הזרם הבלתי מוגדר של TCP.
- לנהל ריבוי לקוחות בו-זמנית עם threads בלי race conditions.
- לממש נכון לחיצת יד הצפנה אסימטרית-סימטרית וגיבוב סיסמאות.
- לבנות מכונת מצבים שפשוט לא מאפשרת פעולה מחוץ לתור.

1.1.2 הגדרת הלקוח

הפרויקט מכון לשני קהלים:

- שחקנים - כל אחד שרוצה משחק קלפים חברתי ברשת מקומית, בלי תשלום אמיתי.
- מורים ותלמידים בחלופת סייבר - להדגמה חיה של הצפנה, threads, sockets ואחסון מאובטח.

1.1.3 יעדים ומטרות

- כמה שחקנים יכולים לשחק ביחד סביב אותו שולחן, בו-זמנית.
- כל שחקן יכול להירשם, להתחבר ולראות את הפרופיל שלו (קרדיטים, ניצחונות, הפסדים, יחס W/L).
- כל התקשורת בין הלקוח לשרת מוצפנת ומאומתת - לא ניתן לקרוא אותה ב-tcpdump.
- לקוח זדוני לא יכול להשפיע על שחקנים אחרים ולא לזייף יתרה.

- ממשק פשוט וברור - גם מישהו שלא יודע כלום יוכל להתחיל לשחק.
- פרופילים נשמרים בין הפעלות - השרת יכול להיסגר ולהיפתח מחדש בלי לאבד נתונים.

1.1.4 בעיות, תועלות וחסכוניות

- הבעיה: רוב משחקי הקלפים המקוונים הם "קופסה שחורה" סגורה, או דורשים תשתית ענן ותשלום חודשי.
- תועלות:
 - משחק חברתי ברשת מקומית, ללא הרשמה לאף שירות חיצוני.
 - סביבת לימוד מצוינת להבנת הצפנה, threads ו-sockets על TCP גולמי.
 - פרופילים נשמרים בין הפעלות - לא מאבדים את הנתונים כשסוגרים את השרת.
 - שירותים שהמערכת מספקת: הרשמה/התחברות, הימור, חלוקת קלפים, Hit/Stand/Double, משחק - דילר אוטומטי, חישוב תוצאות ותשלום, צ'אט, ורענון מצב שולחן בזמן אמת לכל שחקן.

1.1.5 סקירת פתרונות קיימים

חסרונות מבחינתנו	יתרונות	מערכת קיימת
סגור, דורש תשלום, אין גישה לקוד	עיצוב מקצועי	Online Casino (888, PokerStars)
חד-שחקן, ללא רשת, ללא אבטחה	קוד פתוח, פשוט	Blackjack ב-Python (CLI)
מסתיר את שכבות ה-TCP, לא מלמד sockets	רב-משתתפים, נגיש	WebSocket Multiplayer בדפדפן

הפרויקט שלי שונה: רב-משתתפים אמיתי על TCP גולמי, עם לחיצת יד מוצפנת שכתבתי ידנית ושמירת פרופילים ב-SQLite מקומי - כך שאפשר לקרוא ולהבין כל שכבה בעצמך.

1.1.6 סקירת טכנולוגיית הפרויקט

הכול ב-Python 3.11. מרבית הספריות מגיעות עם Python (socket, threading, sqlite3, struct, cryptography (RSA + Fernet), secrets, hmac, hashlib, tkinter). שתי ספריות חיצוניות בלבד: google-genai (Gemini) לסינון צ'אט.

מה לא מתאים לפרויקט הזה:

- הוא לא בשביל אינטרנט פתוח - אין תעודת CA ואין הגנת DDoS מתוחכמת. מיועד לרשת מקומית.
- X. ללא headless דורש מערכת הפעלה עם תמיכה גרפית - לא יעבוד על שרת Tkinter.

1.1.7 תיחום הפרויקט

מה יש בפרויקט:

- תקשורת TCP sockets ופרוטוקול הודעות מותאם.
- ריבוי לקוחות מקביל עם threads.
- הצפנה היברידית (RSA-2048 + Fernet/AES-128) ולחיצת יד מלאה.
- גיבוב סיסמאות (PBKDF2-HMAC-SHA256 + salt) ייחודי לכל משתמש.
- מסד נתונים SQLite לשמירת פרופילים.
- ממשק גרפי ב-Tkinter לשחקנים ולמנהל השרת.
- מכונת מצבים ל-5 שלבי הסבב.

- כסף אמיתי או תשלומים.
- שרתים בענן או מבנה מבוזר.
- דפדפן / WebSockets / אפליקציה מובייל.
- גרפיקה מיוחדת מעבר לקלפים שמצוירים ב-Tkinter.

1.2 פירוט תיאור המערכת (אפיון)

1.2.1 תיאור מפורט של המערכת

המערכת מורכבת משני יישומים נפרדים:

- שרת בלאק-ג'יק - בריצה ראשונה יוצר זוג מפתחות RSA ומאזין על פורט 5050. לכל לקוח - שמתחבר נפתח thread נפרד.
- לקוח בלאק-ג'יק - חלון Tkinter שמתחבר לשרת, מבצע לחיצת יד מוצפנת, ואז עובר למסך - ההתחברות ומשם לשולחן.

1.2.2 מה השחקן יכול לעשות

יש סוג משתמש אחד - שחקן. אלה הפעולות שהוא יכול לבצע:

- להירשם (נפתח פרופיל עם 10,000 קרדיטים).
- להתחבר לשרת קיים.
- להצטרף לשולחן ולשחק עם השחקנים האחרים.
- להמר בתחילת כל סבב.
- לבחור Hit, Stand, או Double בתורו.
- לראות את קלפי הדילר ושל שחקנים אחרים בשולחן.
- לעקוב אחר הפרופיל שלו - קרדיטים, ניצחונות, הפסדים וחס W/L.
- לשוחח בצ'אט עם שחקנים אחרים.
- להתנתק.

1.2.3 בדיקות מתוכננות (קופסה שחורה)

אופן הביצוע	מטרת הבדיקה	#
לעודד אישור ורשומה - register("alice","secret123")	הרשמה תקינה	1
לרשום "alice" פעמיים - לקבל שגיאה בפעם השנייה	הרשמה כפולה	2
להזין סיסמה לא נכונה - לקבל הודעה גברית	סיסמה שגויה	3
להתחבר עם "alice" פעמיים - החיבור השני נדחה	התחברות כפולה	4
הימור, Hit/Stand, דילר, תשלום נכון	משחק מלא לשני שחקנים	5
לבקש הימור גדול מהקרדיטים - שגיאה, היתרה לא	הימור מעל יתרה	6
ללחוץ Hit כשזה לא התור שלך - נדחה בשרת	פעולה מחוץ לתור	7
לא קריאות handshake על הפורט - חבילות לא	האקט	8
לשלוח מסגרת <64 KB - החיבור נסגר, השרת	מחיצה ענקית	9

10	שחזור פרופיל אחרי restart	לעצור ולהפעיל שוב - קרדיטים וניצחונות נשמרו
11	מנוע משחק באמצע סבב	לסגור לקוח בזמן תורו - התור עובר הלאה אוטומטית
12	ולידציית שם משתמש	לנסות שם עם תווים מיוחדים - נדחה

1.2.4 לוח זמנים

מס' בפועל	משך מתוכנן	אבן דרך	#
שבוע 1-2	שבוע 1-2	אפיון, לימוד sockets ו-threads	1
שבוע 3-5 (חריגה בגלל debug של encoding)	שבוע 3-5	פרוטוקול ולחיצת יד	2
שבוע 6-7	שבוע 5-6	מנוע משחק (Card/Deck/Hand/Table)	3
שבוע 7-8	שבוע 7	ניהול פרופילים והצפנת סיסמאות	4
שבוע 8-9	שבוע 8	שילוב שרת ולקוח, threads, broadcasts	5
שבוע 9-11	שבוע 9-10	GUI ב-Tkinter	6
שבוע 11-12	שבוע 11	בדיקות אינטגרציה ותיקון באגים	7
שבוע 12	שבוע 12	כתיבת תיקי הפרויקט	8

1.2.5 ניהול סיכונים

מה בוצע	תכנון	סיכון
כל פעולה משנה-מצב: ProfileManager וב-Table ל-RLock משחק RLock	Race conditions	
החיבור נסגר בנימוס; הלקוח מציג הודעת שגיאה	סגירת חיבור + try/except	כשל בלחיצת היד
ממומש ב-ProfileManager עם PRAGMA journal_mode=WAL	SQLite WAL mode - PRAGMA journal_mode=WAL	שחיתות נתונים בכתיבה
ממומש ב-protocol.py	מגבלת 64 KB לפני קריאה	לקוח שולח הודעה ענקית
ממומש ב-AppController._on_server_message / _process	queue + rootAppController	Tkinter thread-safe לא

1.3 תחום הידע - פירוט יכולות

1.3.1 יכולות בצד השרת

יכולת: קבלת חיבור חדש וניהולו בתהליכון נפרד

- מהות: השרת מאזין על פורט קבוע. לכל לקוח נפתח ClientHandler נפרד. -
- פעולות: socket.accept, threading.Thread, ניהול רשימת מטפלים, סגירה מסודרת. -
- אובייקטים: BlackjackServer, ClientHandler. -

יכולת: לחיצת יד הצפנה (RSA → Fernet)

- מהות: השרת שולח מפתח ציבורי, הלקוח מגריל מפתח Fernet, מצפין אותו ב-RSA ושולח. מכאן - כל הודעה מוצפנת סימטרית.
- פעולות: טעינת/יצירת זוג RSA, פענוח OAEP, יצירת SecureChannel. -
- אובייקטים: RSAKeyPair, SecureChannel. -

יכולת: רישום משתמש חדש

- מהות: קליטת שם וסיסמה, ולידציה, גיבוב ושמירה.
- פעולות: ולידציה, INSERT, hash_password (PBKDF2), commit ב-SQLite.
- אובייקטים: UserProfile, ProfileManager.

יכולת: התחברות משתמש קיים

- מהות: השוואת סיסמה כנגד הגיבוב השמור, עמידות ב-timing attacks, מניעת התחברות כפולה.
- פעולות: verify_password עם hmac.compare_digest; השהיה דמה אם המשתמש לא קיים.
- אובייקטים: ClientHandler, ProfileManager.

יכולת: ניהול שולחן הבלאק-ג'ק

- מהות: מכונת מצבים WAITING → BETTING → PLAYING → DEALER → RESULTS.
- פעולות: קבלת שחקן, קבלת הימור, חלוקת קלפים, מעבר תור, משחק דילר, חישוב תוצאות, איפוס.
- אובייקטים: Table, Player, Dealer, Deck, Hand, Card, Phase.

יכולת: שידור מצב השולחן (broadcast)

- מהות: לאחר כל אירוע, השרת בונה תמונת מצב לכל שחקן (מסתיר קלף נעלם של דילר) ושולח לו.
- פעולות: סריאליזציה של Table, החלפת קלף נעלם, איטרציה על ClientHandler-ים.
- אובייקטים: ClientHandler.send, Table.snapshot_for, BlackjackServer.

יכולת: תשלום וזיכוי לפי תוצאת הסבב

- מהות: השוואת ידיים, תשלום (1:1, 3:2 לבלאק-ג'ק, החזר בפוש), עדכון פרופיל.
- פעולות: record_result, adjust_credits, ProfileManager, Table.resolve_locked.
- אובייקטים: Table, ProfileManager.

יכולת: סינון הודעות צ'אט (Gemini AI)

- מהות: לפני שידור הודעת צ'אט, נשלחת ל-Gemini 2.5 Flash. אם נחסמה - רק השולח מקבל שגיאה. אם ה-API לא זמין - ההודעה עוברת.
- פעולות: בדיקת GEMINI_API_KEY, קריאת google.genai, פרשנות ALLOW/BLOCK.
- אובייקטים: ChatModerator, ClientHandler.

יכולת: לוח בקרה גרפי לשרת

- מהות: חלון Tkinter בזמן אמת: IP/פורט, רשימת שחקנים מחוברים, Kick, לוג צבעוני, שלב המשחק, ועצירת השרת.
- פעולות: Tk window, root.after () כל 400ms, גישה ל-online_ תחת lock.
- אובייקטים: ServerApp, PlayerRow, BlackjackServer.

1.3.2 יכולות בצד הלקוח**יכולת: התחברות לשרת ולחיצת יד**

- מהות: פתיחת socket, פענוח server_hello, יצירת מפתח Fernet, הצפנה ב-RSA ושליחה.

- פעולות: `socket.connect`, `SecureChannel.generate`, `encrypt_session_key`.
- אובייקטים: `ServerConnection`, `SecureChannel`.

יכולת: מסך התחברות/הרשמה

- מהות: שדות שם וסיסמה, שני כפתורים (Login ו-Register), תשובת השרת מוצגת מתחת.
- פעולות: `ttk.Frame`, שליחת בקשה, טיפול בשגיאה.
- אובייקטים: `ConnectFrame`, `LoginFrame`, `AppController`.

יכולת: מסך השולחן

- מהות: אזור דילר, אזור שחקנים עם הדגשה ל"you", פרופיל אישי בסרגל עליון, שדה הימור, כפתורי פעולה.
- פעולות: ציור קלף (`card_label`), איפשור/נטרול כפתורים לפי שלב (`refresh_buttons`).
- אובייקטים: `TableFrame`.

יכולת: רענון אסינכרוני של מצב השולחן

- מהות: הודעות מהרשת מגיעות ב-`thread` רקע. נדחפות לתור, ה-GUI מנקה אותן ב-`main thread` (על `root.after(0, ...)`).
- פעולות: `queue.Queue`, `_on_server_message`, `_process_messages`.
- אובייקטים: `AppController`.

יכולת: שליחת פעולה (Hit/Stand/Double/Bet/Chat)

- מהות: לחיצה מתורגמת להודעה מוצפנת שנשלחת לשרת.
- פעולות: ולידציה לפני שליחה, `ServerConnection.send`, טיפול בכשל רשת.
- אובייקטים: `TableFrame`, `AppController`, `ServerConnection`.

2. מבנה / ארכיטקטורה של הפרויקט

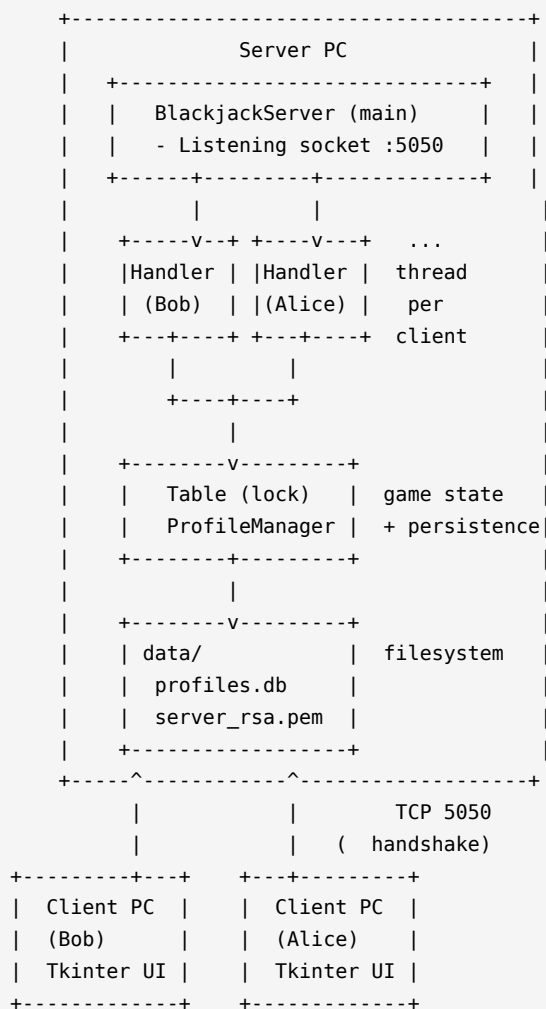
2.1 ארכיטקטורה כללית

הפרויקט בנוי על לקוח-שרת קלאסי עם שרת מרכזי אחד. כל הקלפים, הקופה, הכללים והחשבונות מנוהלים בשרת בלבד. הלקוח מציג את המצב ושולח פעולות - הוא אף פעם לא מקור האמת.

2.1.1 רכיבי חומרה

- מחשב שרת: מריץ `python -m blackjack.run_server`. צריך Python 3.11, חיבור רשת, מקום - לשני קבצים (`profiles.db`, `server_rsa.pem`).
- מחשבי לקוח: כל שחקן מריץ `python -m blackjack.run_client` באותה רשת.
- רשת: TCP/IPv4 על פורט 5050.

2.1.2 שרטוט ארכיטקטורה



2.2 טכנולוגיות בשימוש

סיבה	טכנולוגיה	תחום
קריאה, ספריית סטנדרט עשירה	Python 3.11	שפת תכנות
אמין, מאפשר פרוטוקול מותאם	גולמי, פורט 5050 TCP	תקשורת
תקן מקובל להעברת מפתח	RSA-2048 + OAEP	הצפנה אסימטרית
סודיות + אימות שלמות	Fernet (AES-128-CBC + HMAC-SHA256)	הצפנה סימטרית
קשה ל-brute-force	איטרציות 200,000 PBKDF2-HMAC-SHA256	גיבוב סיסמאות
בנוי ב-Python, ללא תלות	Tkinter	ממשק משתמש
עמיד בפני קריסה, תומך בשאילתות SQL, חלק מ-WAL mode	SQLite (sqlite3) - WAL mode	אחסון
פשוט ומספיק יעיל	threading.Thread + RLock	ריבוי לקוחות
מבין הקשר ושפות	Gemini 2.5 Flash (google-genai)	סינון צ'אט

2.3 זרימת מידע

2.3.1 חיבור שחקן

[Client]	[Server]
TCP connect :5050	
----->	
server_hello {public_key_pem}	
<-----	
key_exchange {encrypted_session_key}	
----->	
ready (encrypted)	
<-----	
login {username, password} (encrypted)	
----->	
	[authenticate]
auth_result {ok, profile} (encrypted)	
<-----	
game_state {snapshot} (encrypted)	
<-----	

2.3.2 זרימת סבב

BETTING:

```
client --place_bet 200--> server --> Table, ProfileManager
server --> broadcast game_state --> all clients
```

PLAYING:

```
Table.deal() -> 2 cards each, dealer hidden card
server -> snapshot for each client
Loop: client --action {hit|stand|double}--> server --> Table
server -> broadcast
```

DEALER:

```
server reveals dealer hole card
while dealer.should_hit(): dealer.add(deck.draw())
server -> broadcast
```

RESULTS:

```
compute outcome, payout, record W/L
server -> round_result + updated profile
Timer (6s) -> next round
```

2.4 אלגוריתמים מרכזיים

2.4.1 חישוב ערך יד עם אסים גמישים

הבעיה: אס שווה 1 או 11 - צריך לבחור את הצירוף שמקסם את הערך בלי לחצות 21.

הערות	סיבוכיות	פתרון
יקר ומסורבל	$O(2^k)$	בדיקה רקורסיבית של כל הצירופים
פשוט, בדיוק מה שקזינואים עושים	$O(n)$	ספירת אסים כ-1, "שדרוג" כל עוד לא חוצים 21

הבחירה: הפתרון השני. ראה `Hand.value` → `common/hand.py`.

2.4.2 ערבוב מאובטח

הבעיה: סדר הקלפים חייב להיות בלתי ניתן לחיזוי.

הבעיה איתו	פתרון
ניתן לחיזוי אחרי 624 ערכים - Mersenne Twister	<code>random.shuffle</code>
מקור אקראי קריפטוגרפי (<code>dev/urandom/</code>), לא ניתן לחיזוי	Fisher-Yates + <code>secrets.randbelow</code>

הבחירה: Fisher-Yates עם `secrets.randbelow`. ראה `Deck.shuffle` → `common/deck.py`.

2.4.3 גיבוב סיסמאות PBKDF2

הבעיה: צריך לאחסן סיסמאות כך שאם מישהו יגנוב את הקובץ, הוא לא יוכל לשחזר אותן.

הבעיה איתו	פתרון
------------	-------

פשוט MD5 / SHA-1	מהיר מדי - מיליוני ניחושים בשנייה, פגיע ל-rainbow
PBKDF2 + salt	200,000 איטרציות - כל ניחוש בודד עולה המון זמן

הבחירה: 200,000 PBKDF2-HMAC-SHA256, salt איטרציות, חלק מ-hashlib. הסטנדרטית. ראה [common/crypto.py](https://pypi.org/project/common/crypto.py).

2.4.4 מסגור הודעות מעל TCP

הבעיה: TCP מספק זרם בתים רציף, לא הודעות מסודרות. צריך לדעת איפה הודעה אחת נגמרת והבאה מתחילה.

הערה	פתרון
עלול להופיע בתוך תוכן מוצפן	delimiter (\\n)
בינארי בטוח, פשוט, מאפשר הגבלת גודל לפני קריאה	length-prefix 4 בתים

הבחירה: 4 length-prefix big-endian בתים + מגבלת 64 KB לפני כל קריאה. ראה [common/protocol.py](https://pypi.org/project/common/protocol.py).

2.4.5 סינון צ'אט בזמן אמת עם Gemini AI

הבעיה: שחקנים יכולים לשלוח הודעות פוגעניות. רשימת מילים שחורות לא מספיקה - קל מדי לעקוף אותה עם ניסוחים יצירתיים.

החיסרון	פתרון
קל לעקוף, לא מבין הקשר	מילים Blocklist
תלות חיצונית נוספת	Perspective API
מבין הקשר, עברית ואנגלית, מהיר, ה-API מוכן	Gemini 2.5 Flash

הבחירה: Gemini 2.5 Flash עם ה-system prompt הבא:

eful, sexually explicit, racist, threatening, or otherwise offensive content. Reply with exactly one word: ALLOW or BLOCK."

אם Gemini מחזיר BLOCK - השרת שולח שגיאה רק לשולח. שאר השחקנים לא רואים כלום. אם ה-API לא זמין - ההודעה עוברת כברירת מחדל (pass-through mode).

קובץ: `(server/chat_moderator.py → ChatModerator → check(message`

```
allowed, reason = _moderator.check(text)
if not allowed:
    self.send("error", {"message": reason})
else:
    self._server.on_chat(self, text)
```

2.5 סביבת פיתוח

Python עם תוסף VS Code Python 3.11 -

חבילות: `pip install cryptography google-genai`.

לניהול גרסאות `git` -

בדיקות ידניות: שרת + שני לקוחות בו-זמנית; `tcpdump` לידוא שהתעבורה מוצפנת; `ls -l` - לידוא הרשאות PEM.

2.6 פרוטוקול תקשורת

2.6.1 שכבות הפרוטוקול

- שכבת מסגור: 4 בתים `big-endian` (אורך) + `n` בתים גוף. מגבלה: $0 < n \leq 65,536$ - כל - ניסיון לשלוח יותר סוגר את החיבור.
- שכבת הצפנה: לפני סיום JSON - `handshake` גלוי. לאחר מכן - פלט Fernet (מוצפן) + HMAC אוטומטי).
- שכבת הודעה: JSON עם המבנה `{"type": "...", "data": {...}}`. סוג ההודעה חייב להיות - מתוך רשימה סגורה - שאר הסוגים נדחים.

2.6.2 פירוט ההודעות

שם הודעה	מ→אל	שדות
server_hello	שרת → לקוח	public_key_pem: str
key_exchange	לקוח → שרת	encrypted_session_key: str (base64)
ready	שרת → לקוח	{}
register	לקוח → שרת	username: str, password: str
login	לקוח → שרת	username: str, password: str
auth_result	שרת → לקוח	ok: bool, message: str, profile?: dict
place_bet	לקוח → שרת	amount: int
action	לקוח → שרת	action: "hit" / "stand" / "double"
game_state	שרת → לקוח	phase, dealer, players, current_username, deck_remaining
round_result	לקוח → שרת	outcome: win/loss/push, payout: int, profile: dict
chat	לקוח → שרת → כולם	message: str (עד 200 תווים, עובר סינון)
error	שרת → לקוח	message: str
logout	לקוח → שרת	{}

2.7 מסכי המערכת

2.7.1 מסך התחברות / הרשמה

שדה Username, שדה Password (תווים מוסתרים), כפתורי Login ו-Register, תווית מצב. -

2.7.2 מסך השולחן

סרגל פרופיל עליון (קרדיטים, ניצחונות, הפסדים, W/L), אזור דילר, אזור שחקנים (הדגשה -

2.7.3 תוצאת סבב

באנר (WIN = זזהב, LOSS = אדום, PUSH = אפור) למשך 6 שניות ואז סבב חדש. -

2.7.4 לוח בקרה לשרת

- כותרת: שם האפליקציה + Host/IP/Port.
- שמאל: רשימת שחקנים מחוברים + כפתור Kick לכל אחד.
- ימין: לוג עם צבע לפי חומרה.
- שורה תחתונה: שלב משחק, קלפים שנותרו, מונה Gemini, כפתור Stop.

2.7.5 תרשים מסכים

```

+-----+ login OK +-----+
| LoginFrame |----->| TableFrame |
+-----+ +-----+
^ |
| disconnect / logout |
+-----+

+-----+ ( )
| ServerApp |
+-----+
    
```

2.8 מבני נתונים

2.8.1 טבלת users - מסד הנתונים profiles.db

מסד הנתונים הוא קובץ SQLite יחיד. הטבלה נוצרת אוטומטית בריצה ראשונה.

```

CREATE TABLE IF NOT EXISTS users (
    username TEXT PRIMARY KEY,
    password_hash TEXT NOT NULL,
    credits INTEGER NOT NULL DEFAULT 10000,
    wins INTEGER NOT NULL DEFAULT 0,
    losses INTEGER NOT NULL DEFAULT 0,
    pushes INTEGER NOT NULL DEFAULT 0
);
    
```

שם עמודה	טיפוס	אורך / טווח	דוגמה
username	TEXT (PRIMARY KEY)	3-16 תווים, _A-Za-z0-9	"alice"
password_hash	TEXT	פורמט salt_b64\$hash_b64	"k4Ja...\$g2Cv..."
credits	INTEGER	>= 0	9750
wins	INTEGER	>= 0	4

losses	INTEGER	>= 0	2
pushes	INTEGER	>= 0	1

כתיבות אטומיות מובנות, עמיד בפני קריסה - WAL (Write-Ahead Log) פועל במצב SQLite

2.8.2 קובץ server_rsa.pem

מפתח RSA פרטי בפורמט PEM. הרשאות 0o600 (קריאה/כתיבה לבעל בלבד). נוצר אוטומטית בריצה ראשונה.

2.8.3 מבני נתונים בזיכרון

- BlackjackServer._clients : List[ClientHandler]
- BlackjackServer._online : Dict[str, ClientHandler]
 - Table._players : List[Player]
 - Deck._cards : List[Card]
- AppController._msg_queue : queue.Queue

2.9 סקירת חולשות ואיומים

2.9.1 שכבת האפליקציה

מה נעשה	רלוונטי?	איום
כל שאילתה משתמשת בפרמטרים (?) - אין שרשרת מחרוזות		SQL Injection
salt; hmac.compare_digest (constant-time)		אימות (login)
RSAPSS + OAEP + Fernet; זור		MITM
thread לכל לקוח; socket		DoS
Parser; קפדני עם רשימה סגורה		Fuzzing
logged_in; snapshot_for הנע		גישה לנתוני שחקן אחר
secrets; random (/dev/urandom),		אקראיות חלשה
רק כמשתנה סביבה; לא בקוד, לא בלוג, לא ב-git		חשיפת מפתח Gemini

2.9.2 שכבת התעבורה

- לחיצת היד המשולשת מנוהלת על ידי מערכת ההפעלה; אין מה להוסיף כאן - TCP
- הצפנה: (hybrid handshake (RSA + Fernet) - סודיות + שלמות, בלי להזדקק ל-TLS מלא.
- מחזיר פחות בתים ממה שביקשנו socket-מבטיח שלא נקרא חצי הודעה, גם כשה-read_exactly

3. מימוש הפרויקט

3.1 חלק א' - תיאור מחלקות וספריות

3.1.1 ספריות חיצוניות

שימוש	ספרייה
RSA-2048 + OAEP	<code>cryptography.hazmat.primitives.asymmetric.rsa</code>
פורמט PEM	<code>cryptography.hazmat.primitives.serialization</code>
הצפנה סימטרית מאומתת	<code>cryptography.fernet.Fernet</code>
קריאות ל-Gemini 2.5 Flash לסינון צ'אט	<code>google.genai</code>

3.1.2 מחלקות שפותחו בפרויקט

Card (common/card.py)

- תפקיד: קלף בודד, `immutable` - לא ניתן לשנות אחרי יצירה.
- תכונות: `rank: str ('A','2'..'K')`, `suit: str ('S','H','D','C')`.
- פעולות: ולידציה; `point_value`; `is_ace`; `to_dict/from_dict`.

Deck (common/deck.py)

- תפקיד: "shoe" של N חבילות מעורבבות.
- תכונות: `cards: list[Card]`, `_num_decks: int`.
- פעולות: `shuffle()` - Fisher-Yates + `secrets.randbelow`; `draw() → Card`; `__len__`.

Hand (common/hand.py)

- תפקיד: יד של שחקן או דילר עם חישוב ערך חכם לאסים.
- פעולות: `add(card)`; `clear()`; `value (property)`; `is_soft`; `is_blackjack`; `is_bust`; `to_list()`.

SecureChannel (common/crypto.py)

- תפקיד: עטיפה ל-Fernet - ערוץ סימטרי מוצפן לכל חיבור TCP.
- פעולות: `generate()` (classmethod); `encrypt(plaintext)`; `decrypt(token)`.

RSAPKeyPair (common/crypto.py)

- תפקיד: מפתחות RSA-2048 של השרת לחילוף מפתח `session`.
- פעולות: `generate()`; `load_or_create(path)`; `public_pem()`; `decrypt_session_key(b64)`.

UserProfile (server/profile_manager.py)

- תפקיד: ייצוג בזיכרון של חשבון שחקן אחד.
- תכונות: `username`, `password_hash`, `credits`, `wins`, `losses`, `pushes`.
- פעולות: `games_played`; `win_loss_ratio`; `to_dict`.

ProfileManager (server/profile_manager.py)

- תפקיד: ממשק ל-SQLite (profiles.db); רישום, אימות, שינוי קרדיטים, עדכון תוצאות - בטוח ל-threads.
- תכונות: conn: sqlite3.Connection, _lock: RLock
- פעולות: register(u,p); authenticate(u,p); adjust_credits; record_result; get _fetch.

Phase (server/game.py)

- תפקיד: enum של חמשת שלבי הסבב - WAITING, BETTING, PLAYING, DEALER, RESULTS.

RoundResult (server/game.py)

- תפקיד: dataclass עם תוצאת סבב לשחקן אחד (username, outcome, payout).

Table (server/game.py)

- תפקיד: מכונת מצבים של שולחן הבלאק-ג'יק - מנהלת שחקנים, הימורים ותורות.
- תכונות: players, _dealer, _deck, _phase, lock: RLock, callbacks_
- פעולות: add_player; remove_player; place_bet; handle_action; _begin_round; _advance_turn; _dealer_turn; _score_round; reset_for_next_round; snapshot_for(username).

ClientHandler (server/client_handler.py) - thread

- תפקיד: thread אחד לכל לקוח - מחזיק socket וסשן מוצפן.
- פעולות: _on_login/register; _handle(); _message_loop(); _handshake(); _run(); send(); shutdown().

BlackjackServer (server/server.py)

- תפקיד: שרת ראשי - מקבל חיבורים, מחזיק שולחן, מתאם broadcasts.
- פעולות: serve_forever(); stop(); is_online; on_player_joined; on_player_left; on_bet; on_action; on_chat; _broadcast; _push_profile.

ServerConnection (client/network.py)

- תפקיד: חיבור מוצפן מהלקוח לשרת, כולל thread האזנה ברקע.
- פעולות: connect(); _handshake(); send(); _listen(); close.

AppController (client/gui.py)

- תפקיד: בקר ראשי - מחבר בין רשת ל-GUI, מנהל מעברי מסך.
- פעולות: _on_server_message; _process_messages; _route_message; _show_login/_show_table; run(); send(); connect; login; register; logout.

ChatModerator (server/chat_moderator.py)

- תפקיד: סינון צ'אט עם Gemini - מחזיר (allowed, reason) לכל הודעה.
- פעולות: (bool, str) → (check(message)).

ServerApp (server/gui.py)

- תפקיד: לוח בקרה גרפי לשרת - שחקנים, לוג, סטטוס משחק.
- פעולות: _poll; _run(); _kick(username); _on_stop_click; _update_player_list(); 400ms; _run().

3.2 קטעי קוד מרכזיים - כאן אפשר לראות את העקרונות בפעולה

3.2.1 חישוב ערך יד עם אסים

```
# blackjack/common/hand.py
@property
def value(self) -> int:
    total = sum(c.point_value for c in self._cards) # all aces start as 1
    aces = sum(1 for c in self._cards if c.is_ace)
    while aces > 0 and total + 10 <= 21:
        total += 10          # promote one ace from 1 to 11
        aces -= 1
    return total
```

3.2.2 ערבוב מאובטח של החבילה

```
# blackjack/common/deck.py
def shuffle(self) -> None:
    self._cards = [
        Card(rank, suit)
        for _ in range(self._num_decks)
        for suit in Card.SUITS
        for rank in Card.RANKS
    ]
    # Fisher-Yates with secrets.randbelow - ,
    for i in range(len(self._cards) - 1, 0, -1):
        j = secrets.randbelow(i + 1)
        self._cards[i], self._cards[j] = self._cards[j], self._cards[i]
```

3.2.3 לחיצת יד הצפנה (צד שרת)

```
# blackjack/server/client_handler.py
def _handshake(self) -> None:
    send_plain(self._sock, "server_hello",
               {"public_key_pem": self._rsa.public_pem()})
    msg = recv_plain(self._sock)
    if msg["type"] != "key_exchange":
        raise ProtocolError("expected key_exchange")
    session_key = self._rsa.decrypt_session_key(
        msg["data"]["encrypted_session_key"])
    self._channel = SecureChannel(session_key)
    send_encrypted(self._sock, self._channel, "ready", {})
```

3.2.4 גיבוב והשוואת סיסמאות

```
# blackjack/common/crypto.py
ITERATIONS = 200_000

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    return f"{base64.b64encode(salt).decode()}$" \
        f"{base64.b64encode(digest).decode()}"

def verify_password(password: str, stored: str) -> bool:
    try:
        salt_b64, hash_b64 = stored.split("$", 1)
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(hash_b64)
    except (ValueError, binascii.Error):
        return False
    actual = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    return hmac.compare_digest(actual, expected) # constant-time
```

3.2.5 מסגור הודעה עם הגנה על גודל

```
# blackjack/common/protocol.py
MAX_MESSAGE_BYTES = 64 * 1024
_LENGTH_PREFIX = struct.Struct(">I")

def send_frame(sock, payload):
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ProtocolError("payload too large")
    sock.sendall(_LENGTH_PREFIX.pack(len(payload)) + payload)

def recv_frame(sock):
    header = _recv_exact(sock, _LENGTH_PREFIX.size)
    (length,) = _LENGTH_PREFIX.unpack(header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ProtocolError(f"invalid frame length: {length}")
    return _recv_exact(sock, length)
```

3.2.6 צילום מצב בטוח לכל שחקן

```
# blackjack/server/game.py
def snapshot_for(self, username: str) -> dict:
    with self._lock:
        dealer_cards = self._dealer.hand.to_list()
        if self._phase is Phase.PLAYING and len(dealer_cards) >= 2:
            dealer_cards = [dealer_cards[0], {"rank": "?", "suit": "?"}]
            value_hidden = True
        else:
            value_hidden = False
        return {
            "phase": self._phase.value,
            "dealer": {"cards": dealer_cards,
                       "value_hidden": value_hidden,
                       "value": self._dealer.hand.value
                       if not value_hidden else None},
            "players": [self._player_dict(p, p.username == username)
                        for p in self._players],
            "current_username": (self.current_player().username
                                if self.current_player() else None),
            "deck_remaining": len(self._deck),
        }
```

3.3 בדיקות

3.3.1 בדיקות שתוכננו מראש

#	מה בדקתי	תוצאה	באגים שנתקלתי בהם
1	עבר שמוחלט לוח	רשומה נוצרה ב-GUI; profiles.db	-
2	רישום כפול של אותו שם	"username already taken"	-
3	כיסוי שם או סיסמה	"Invalid username or password"	בגרסה ראשונה חשפתי "known user"
4	login ששם או סיסמה	"username already logged in"	-
5	קישור הראשון לא הופרע	3:2 על בלאק-ג'ק	בהתחלה שילמתי 1:1 על בלאק-ג'ק
6	קישור מלא לשני שחקנים	"insufficient credits"	-
7	יתרה לא השתנתה	"Not your turn"	-
8	מעל יתרה	פעולה מחוץ לתור	-
9	האקדמי	handshake - רעש בינארי בלתי תקין	-
10	קודקוד	tcpdump עם קודקוד	-
11	ענקה (KB 200)	ענקה	בודק את הגודל לפני הקצאת מאגר
12	הפעלה מחדש של השרת	קרדיטים וניצחונות נשמרו	-
13	התור עבר אוטומטית הלאה; הימור נחשק	on_player_left	נוסף טיפול ב-
14	התור עבר אוטומטית הלאה; הימור נחשק	only letters, digits and underscore	נדחה: "only letters, digits and underscore"

3.3.2 בדיקות שנוספו בהמשך

#	מה בדקתי	תוצאה	באגים שנתקלתי בהם
13	אוטומטי end-to-end	עבר לחלוטין	-
14	הרשאות קובץ PEM	644; 660; 777	בהתחלה נשמר עם 644; נוסף 660; אחרי-ההגדרה

15	handshake לא תקין אחרי JSON	ProtocolError; החיבור נסגר	-
16	10 לקוחות בו-זמנית	handshake; CPU <5% עברו	-
17	בדיקת אקראיות (secrets vs random)	אין דפוס ניתן לחיזוי	-
18	Connection lost; שיתוף פתאומי	חזרה למסך ההתחברות	-

4. מדריך למשתמש - איך מריצים את הפרויקט

4.1 עץ קבצים

```

blackjack/
├── README.md
├── .md
├── run_server.py          <-  ()
├── run_server_gui.py     <-
├── run_client.py
├── common/
│   ├── card.py
│   ├── deck.py
│   ├── hand.py
│   ├── protocol.py
│   └── crypto.py
├── server/
│   ├── profile_manager.py
│   ├── game.py
│   ├── client_handler.py
│   ├── chat_moderator.py
│   ├── gui.py
│   └── server.py
├── client/
│   ├── network.py
│   └── gui.py
└── data/                  <-  (gitignored)
    ├── profiles.db
    └── server_rsa.pem
  
```

4.2 התקנה

4.2.1 דרישות

- (ומעלה) (מומלץ 3.10 3.11 Python -
- עם ממשק גרפי Linux / macOS / Windows
- זמין pip
- רשת מקומית שבה השרת והלקוחות יכולים לתקשר.

4.2.2 התקנת חבילות

```
pip install cryptography google-genai
```

בלינוקס, אם הוא לא מותקן. Python מגיע עם Tkinter

```
sudo apt install python3-tk
```

סינון הצ'אט עם Gemini הוא אופציונלי לחלוטין – אם לא מגדירים מפתח, כל הודעה עוברת. כדי להפעיל:

```
export GEMINI_API_KEY="< >" # Linux / macOS
set GEMINI_API_KEY=< > # Windows
```

4.2.3 מה קורה בפעם הראשונה

- אין חשבונות מוגדרים מראש – כל שחקן נרשם בעצמו.
- כל חשבון חדש מקבל 10,000 קרדיטים.
- גבולות הימור: 50 עד 5,000 קרדיטים לסבב.

4.2.4 הגדרת כתובת הרשת

- ברירת מחדל: שרת מאזין על 0.0.0.0:5050; לקוח מתחבר ל-127.0.0.1:5050.
- לחיבור ממחשב אחר:

```
BJ_HOST=192.168.1.10 BJ_PORT=5050 python -m blackjack.run_client
```

4.2.5 דרישות חומרה

מינימום: מחשב 1 GB RAM, 512 MHz, פחות מ-5 MB מקום פנוי. כמעט כל מחשב יעבוד.

4.3 הפעלה

4.3.1 הפעלת השרת

א. מצב טרמינל (פשוט):

```
python -m blackjack.run_server
```

ב. מצב לוח בקרה גרפי (מומלץ):

```
python -m blackjack.run_server_gui
```

לעצירה: Ctrl+C בטרמינל, או כפתור "Stop Server" בלוח הבקרה.

4.3.2 הפעלת הלקוח

```
python -m blackjack.run_client
```

נפתח חלון עם שדה הכתובת. מזינים את IP השרת, לוחצים Join, ואז נרשמים או מתחברים.

4.3.3 תיאור מסכים

- מסך חיבור לשרת: כתובת ופורט. ברירת מחדל: 127.0.0.1:5050.
- מסך התחברות: שם וסיסמה → Register (בפעם הראשונה) או Login.
- שולחן - תחילת סבב: הזן הימור ולחץ Place Bet.
- שולחן - התור: Hit לקלף נוסף, Stand לעצירה, Double להכפלת הימור.
- שלב דילר: הקלף הנעלם נחשף; הדילר מושך קלפים אוטומטית לפי הכללים.
- תוצאה: באנר זהב = ניצחת; אדום = הפסדת; אפור = פוש (החזר הימור).
- לוח בקרה לשרת: רשימת שחקנים + Kick, לוג צבעוני ושלב המשחק הנוכחי.

5. סיכום אישי / רפלקציה

עבודה על הפרויקט הייתה מסע ארוך. בתחילה היה לי רעיון מעורפל - "אעשה משחק קלפים עם רשת" - אבל מהר מאוד גיליתי שכל מילה מסתירה עולם של החלטות.

הצלחות:

הרגע שהפעלתי שני לקוחות בו-זמנית והם ראו אותו שולחן - threads הפכו מתיאוריה לכלי - עבודה.

כש-tcpdump הראה שהתעבורה לא קריאה אחרי handshake - ההצפנה עבדה. -
כשהבנתי לבד למה random לא מתאים לקזינו ועברתי ל-secrets. -

אתגרים:

הבדל בין bytes למחרוזות ב-Python 3 - שעות על TypeError. -
באותו רגע Hit בגרסה ראשונה לא תמיד ניתן היה להמר כשמישהו אחר שלח Race conditions -
הפתרון: threading.RLock על כל פעולה משנה-מצב.

למידה:

רוב הלמידה הייתה עצמאית - תיעוד Python, Real Python, Computerphile. -
בכל פעם שנתקעתי כתבתי לעצמי "מה בדיוק אני מנסה להעביר" - הטקסט הזה לבדו פתר חצי - מהבאגים.

הרגלים שאקח הלאה:

להגדיר פרוטוקול תקשורת בכתב לפני שכותבים שורת קוד. -
להשתמש ב-logging מסודר ולא ב-print. -
להפריד שכבות - common, server, client - כך שכל קובץ קריא לבד. -

במבט לאחור - מה הייתי משנה:

הייתי בוחר asyncio במקום threading לניהול ריבוי לקוחות. -
הייתי מוסיף pytest עם בדיקות יחידה מההתחלה. -

שאלות שנשארו פתוחות:

האם secrets.randbelow עמיד לחלוטין, או שיש מתקפות על urandom? -
איך עובד OAEP מתחת למכסה המנוע? -
איך קזינואים אמיתיים מוודאים שהדילר שלך קלף מהצמרת? -

6. ביבליוגרפיה

- Python Software Foundation. (2024). socket — Low-level networking interface.
<https://docs.python.org/3/library/socket.html>
 - Python Software Foundation. (2024). threading — Thread-based parallelism.
<https://docs.python.org/3/library/threading.html>
 - Python Software Foundation. (2024). secrets — Generate secure random numbers.
<https://docs.python.org/3/library/secrets.html>
 - Python Software Foundation. (2024). hashlib — Secure hashes and message digests.
<https://docs.python.org/3/library/hashlib.html>
 - Python Cryptographic Authority. (2024). Cryptography documentation.
<https://cryptography.io/en/latest/>
 - Bellare, M., & Rogaway, P. (1995). Optimal asymmetric encryption — How to encrypt with RSA. EUROCRYPT '94.
 - OWASP Foundation. (2023). Password Storage Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
 - Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson.
 - Computerphile. (2017). Public Key Cryptography [Video].
https://www.youtube.com/watch?v=GSIDS_lvRv4
 - Google. (2025). Google Generative AI Python SDK (google-genai).
<https://ai.google.dev/gemini-api/docs/sdks>
-

7. נספחים

נספח א' - הסבר קצר על הטכנולוגיות

B.1 RSA-2048 + OAEP

היא הצפנה אסימטרית: מפתח ציבורי לכולם, מפתח פרטי רק לשרת. מה שמוצפן עם הציבורי - רק RSA הפרטי יכול לפענח. בפרויקט הזה RSA משמש פעם אחת בלבד לכל חיבור - להעברת מפתח ה-Fernet. RSA מוסיף אקראיות לכל הצפנה ומונע מתקפות סטטיסטיות על OAEP.

B.2 Fernet

מפרט הצפנה המבוסס על AES-128-CBC + HMAC-SHA256. כל שינוי בבית אחד בחבילה מוביל לכישלון פענוח. כולל timestamp שמאפשר לגלות הודעות "ישנות" שנשלחו מחדש.

B.3 PBKDF2

הפעלה חוזרת ומכוונת של HMAC על הסיסמה ו-200,000 salt. איטרציות הופכות כל ניחוש בודד לפעולה יקרה חישובית - מה שהופך brute-force לבלתי כדאי.

B.4 TCP

מקנה זרם בתים אמין בין שני קצוות הרשת. לא כולל מסגור הודעות - את זה אנחנו בונים בעצמנו עם length-prefix.

B.5 Threads ו-Locks

אחד thread מבטיח שרק RLock (Reentrant Lock) הוא נתיב ביצוע מקביל בתוך אותו תהליך Thread יכול לשנות נתון מסוים בכל רגע נתון, ומונע race conditions.

B.6 Gemini AI

מודל שפה גדול של Google. בפרויקט, Gemini 2.5 Flash מסווג כל הודעת צ'אט כ-ALLOW או BLOCK תוך שניות. אם ה-API לא מוגדר או לא זמין, ChatModerator פשוט מעביר את ההודעה הלאה בלי לחסום.

נספח ג' - קוד מלא (לצירוף בגרסה מודפסת)

יש לצרף את הקבצים בסדר הבא:

- common/card.py
- common/deck.py
- common/hand.py

- common/protocol.py
- common/crypto.py
- server/profile_manager.py
 - server/game.py
- server/client_handler.py
 - server/server.py
- client/network.py
 - client/gui.py
- server/chat_moderator.py
 - server/gui.py
- run_server.py
- run_server_gui.py
 - run_client.py